

Practica 2

Víctor Manuel Peiró Martínez

FIIS 3A

Programación de Sistemas Distribuidos

Contenido

Introducción.....	3
Docker	3
Kubernetes	3
Archivos YAML	4
Archivos del FileManager	4
Configuración Básica	5
Configuración Servidor	5
Docker	5
Kubernetes	6
Configuración Broker	7
Docker	7
Kubernetes	7
Configuración Cliente	8
Docker	8
Kubernetes	9
Ejecución	9
Configuración AWS.....	9
Creación de Pods	10
Configuración Avanzada 1.....	11
Montaje de Volúmenes	11
Configuración Avanzada 2.....	13
NFS.....	13
Montaje Servidor NFS.....	14
PersistentVolume	15
PersistentVolumeClaim	16
Modificación Servidor.....	17

Introducción

El objetivo de la práctica es distribuir un sistema de ficheros usando kubernetes y Docker. Para ello, tendremos 3 componentes principales:

- Servidores
- Clientes
- Broker → Redirigen usuarios a servidores

Docker

Con docker, crearemos imágenes que serán usadas por los pods para hacer funciones específicas. Para ello, serán útiles los siguientes comandos:

- **docker build -t <usuario>/<nombre_imagen>:** Permite construir imágenes de docker
- **docker images:** Permite ver todas las imágenes de Docker
- **docker push docker.io/<usuario>/<nombre_imagen>:latest:** Permite subir una imagen a Docker Hub

A cada imagen le corresponde un Dockerfile conteniendo las funciones de la imagen.

Kubernetes

Usaremos Kubernetes para gestionar un clúster formado por varias máquinas (nodos), donde cada nodo ejecutará diferentes componentes de nuestra aplicación. Cada nodo puede ejecutar uno o varios pods, y cada pod contiene uno o más contenedores que ejecutan código específico. Gracias a Kubernetes podremos desplegar, replicar y administrar estos pods de forma automática dentro del clúster. Para ello, los siguientes comandos serán útiles:

- **kubectl get nodes:** Devuelve los nodos añadidos a kubernetes
- **kubectl get pods:** Devuelve todos los pods
- **kubectl describe pods:** Devuelve información sobre todos los pods
- **kubectl apply -f <nombre_archivo>.yaml:** Permite desplegar pods con una configuración descrita en un archivo .yaml
- **kubectl exec -ti <nombre-del-pod> -- <comando>:** Ejecuta un comando en un pod
- **bash kub_addNode.sh <IP_maquina>:** Añade un nodo esclavo

Archivos YAML

Para configurar los pods que desplegaremos en el clúster, utilizaremos archivos de configuración en formato YAML. En Kubernetes trabajaremos principalmente con dos tipos de archivos:

1. **Deployment**, que define cómo se crean y gestionan los pods (réplicas, imagen Docker, variables, puertos...).
2. **Service**, que expone esos pods dentro o fuera del clúster y les proporciona una dirección estable para permitir su comunicación.

Algunas definiciones de estos archivos contienen la siguiente información:

- **apiVersion**: Define la versión de la API de Kubernetes
- **kind**: Define el tipo de objeto creado. En nuestro caso, Deployment o Service
- **metadata**: Permite definir metadatos del objeto que creamos
- **spec**: Permite definir cosas del objeto
- **replicas**: Numero de pods que se crearan
- **selector**: Sirve para unir el deployment/service con los pods creados
- **template**: Sirve para definir la plantilla del pod
- **containers**: Lista de contenedores dentro del pod donde se definirán cosas como la imagen, nombre, variable de entorno etc
- **image**: Permite definir la imagen del pod
- **env**: Permite definir variables de entorno
- **type: NodePort** : Expone el servicio hacia fuera del cluster
- **port**: Puerto Interno del servicio
- **targetPort**: Puerto del contendor

Archivos del FileManager

Se nos ha proporcionado 3 ejecutables con las funcionalidades para cada componente siendo estas:

- **clientFileManager** para funcionalidad del cliente. Recibe por parámetros la IP del Broker
- **brokerFileManager** para funcionalidades del broker
- **serverFileManager** para funcionalidades del servidor. Recibe por parámetros la IP del Broker

Configuración Básica

Para la configuración básica usaremos lo siguiente:

- Nodo Master con el control pane de la aplicación
- Nodo Broker con el deployment de brokerFileManager
- Nodo Esclavo con el deployment de serverFileManager

Configuración Servidor

Para configurar el servidor debemos tener en cuenta lo siguiente:

- El programa serverFileManager usa el puerto 32001
- Se creará más de un servidor

Docker

A continuación, se mostrará el contenido del Dockerfile para la creación de la imagen

```
FROM ubuntu:22.04

RUN apt-get update
RUN apt-get install -y libstdc++6 software-properties-common
RUN apt-get clean

WORKDIR /app

RUN mkdir -p /app/FileManagerDir
RUN touch /app/FileManagerDir/test.txt

COPY serverFileManager /app/serverFileManager

RUN chmod +x /app/serverFileManager

EXPOSE 32001

ENTRYPOINT ["/serverFileManager"]
CMD ["44.215.164.220","32002","44.215.164.220","32001"]
```

- **FROM ubuntu:22.04:** Establece la imagen base
- **RUN apt-get update:** Actualiza la lista de paquetes de la imagen
- **RUN apt-get install -y libstdc++6 software-properties-common:** Instala la librería estándar de C++ y la librería software-properties-common
- **WORKDIR /app:** Define el directorio /app como directorio de trabajo
- **RUN apt-get clean:** Limpia el cache de paquetes
- **COPY serverFileManager /app/serverFileManager:** Copia el fichero en el WORKDIR
- **RUN mkdir -p FileManagerDir:** Crea el directorio en el contenedor
- **RUN touch /app/FileManagerDir/test.txt:** Crea fichero de prueba
- **RUN chmod +x serverFileManager:** Otorga permisos de ejecución al fichero
- **EXPOSE 32001:** Documenta el puerto que el contenedor escucha
- **ENTRYPOINT ["/serverFileManager"]:** Comando a ejecutar
- **CMD ["", "", "", ""]:** Parametros de comando a ejecutar

Kubernetes

Deployment YAML:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: server
spec:
  replicas: 3
  selector:
    matchLabels:
      app: server
  template:
    metadata:
      labels:
        app: server
    spec:
      containers:
        - name: server
          image: docker.io/19peirooo/server:latest
          ports:
            - containerPort: 32001
```

Service YAML:

```
apiVersion: v1
kind: Service
metadata:
  name: server-service
spec:
  type: NodePort
  selector:
    app: server
  ports:
    - port: 32001
      nodePort: 32001
  externalTrafficPolicy: Cluster
```

externalTrafficPolicy: Define como se enruta el tráfico externo hacia los pods

Configuración Broker

Para configurar el docker se tendrá en cuenta:

- El programa brokerFileManager usa el puerto 32002

Docker

```
FROM ubuntu:22.04

RUN apt-get update
RUN apt-get install -y libstdc++6 software-properties-common
RUN apt-get clean

WORKDIR /app

COPY brokerFileManager /app/brokerFileManager
RUN chmod +x /app/brokerFileManager

EXPOSE 32002

ENTRYPOINT ["/app/brokerFileManager"]
```

ENTRYPOINT ["/app/brokerFileManager"]: Se ejecutará el fichero brokerFileManager en el docker

Kubernetes

Deployment YAML:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: broker
spec:
  replicas: 1
  selector:
    matchLabels:
      app: broker
  template:
    metadata:
      labels:
        app: broker
    spec:
      containers:
        - name: broker
          image: docker.io/19peirooo/broker:latest
          ports:
            - containerPort: 32002
```

Service YAML:

```
apiVersion: v1
kind: Service
metadata:
  name: broker-service
spec:
  type: NodePort
  selector:
    app: broker
  ports:
  - protocol: TCP
    port: 32002
    nodePort: 32002
    targetPort: 32002
  externalTrafficPolicy: Cluster
```

Configuración Cliente

Docker

```
FROM ubuntu:22.04

RUN apt-get update
RUN apt-get install -y libstdc++6

WORKDIR /app

COPY clientFileManager /app/clientFileManager
RUN chmod +x /app/clientFileManager

CMD ["sleep", "infinity"]
```


Kubernetes

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: client
spec:
  replicas: 1
  selector:
    matchLabels:
      app: client
  template:
    metadata:
      labels:
        app: client
    spec:
      containers:
        - name: server
          image: docker.io/19peirooo/client:latest
```

Ejecución

Configuración AWS

En AWS configuraremos 2 maquinas EC2 cada una con su IP publica elástica para que esta no cambie.

Instancias (2) Información

Última actualización
Hace 1 minute

Conectar

Estado de la instancia

Acciones

Q

Buscar instancia por atributo o etiqueta (case-sensitive)

En ejecución

☐	Name	ID de la instancia	Estado de la i...	Tipo de inst...	Comprobación de	Estado de la al	Zona de dispon...	DNS de IPv4 pública	Dirección IP...	IP elástica
☐	Maquina1	i-0ef4ea202502f1053	En ejecución	t3a.medium	3/3 comprobaci...	Ver alarmas +	us-east-1d	ec2-54-243-129-15.co...	54.243.129.15	54.243.129.15
☐	Maquina2	i-00a76cdea720bf97	En ejecución	t3a.medium	3/3 comprobaci...	Ver alarmas +	us-east-1c	ec2-44-215-164-220.co...	44.215.164.220	44.215.164.220

Estas máquinas están vinculadas por un grupo de seguridad donde se crean las siguientes reglas para permitir el tráfico:

Reglas de entrada (8)									
Administrar etiquetas Editar reglas de entrada									
Buscar									
☐	Name	ID de la regla del gr...	Versión de IP	Tipo	Protocolo	Intervalo de puertos	Origen	Descripción	
☐	-	sgr-089883946e457846c	IPv4	TCP personalizado	TCP	8080	0.0.0.0/0	-	
☐	-	sgr-0c91836b7cc63744f	IPv4	TCP personalizado	TCP	6443	0.0.0.0/0	-	
☐	-	sgr-05f69a4d07b75958c	IPv4	TCP personalizado	TCP	10259	0.0.0.0/0	-	
☐	-	sgr-00429f758cfc6bcb	IPv4	SSH	TCP	22	0.0.0.0/0	-	
☐	-	sgr-059a254616a81a5eb	IPv4	TCP personalizado	TCP	10250	0.0.0.0/0	-	
☐	-	sgr-07d0fb7c3013cb3a3	IPv4	TCP personalizado	TCP	30000 - 32767	0.0.0.0/0	-	
☐	-	sgr-046189c03111194db	IPv4	TCP personalizado	TCP	2379 - 2380	0.0.0.0/0	-	
☐	-	sgr-05d61a778524e047a	IPv4	TCP personalizado	TCP	10257	0.0.0.0/0	-	

Creación de Pods

Lo primero que hay que hacer es añadir un nodo esclavo. Para ello, se ejecuta comando mencionado anteriormente con la IP 44.215.164.220 siendo esta la IP de la maquina 2.

Después subiremos las imágenes a docker hub con lo siguiente:

```
~/Practica2/kub $ docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
19peirooo/broker:latest	7e9cbf49c551	428MB	124MB	
19peirooo/client:latest	a3c0e50bf300	237MB	78MB	
19peirooo/practica2/broker:latest	32c560ddac2f	112MB	28.2MB	
19peirooo/server:latest	4a422045ed4e	428MB	124MB	
localserver:latest	e307ff17a21a	177MB	55.1MB	U

Después hay que crear todos los pods

```
~/Practica2/kub $ kubectl apply -f deployment-broker.yaml
deployment.apps/broker created
~/Practica2/kub $ kubectl apply -f deployment-server.yaml
deployment.apps/server created
~/Practica2/kub $ kubectl apply -f deployment-client.yaml
deployment.apps/client created
~/Practica2/kub $ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
broker-745d76fcf8-nrmt5	1/1	Running	0	19s
client-65b494589b-vjjx6	1/1	Running	0	5s
server-d4dc895b-5xj9p	1/1	Running	0	11s
server-d4dc895b-9m4kr	1/1	Running	0	11s
server-d4dc895b-m654t	1/1	Running	0	11s

Una vez lanzados todos los pods para hacer las pruebas usaremos el server-d4dc895b-5xj9p como servidor de pruebas. Ahora nos meteremos en el cliente para probar si funciona todo:

```
~/Practica2/kub $ kubectl exec -ti client-65b494589b-vjjx6 -- bash
root@client-65b494589b-vjjx6:/app# ls
clientFileManager
root@client-65b494589b-vjjx6:/app# ./clientFileManager 44.215.164.220 32002
Enter command:
ls
Listing files in local path
"./clientFileManager"
Enter command:
lls
Listing files fileManager path
FileManagerDir/test.txt
Enter command:
exit()
Exiting system
root@client-65b494589b-vjjx6:/app#
```

Configuración Avanzada 1

Para esta implementación, tendremos que coger lo ya implementado y añadir una carpeta compartida entre todos los nodos que deberá usar serverFileManager para que cuando se conecte un cliente vea siempre la misma carpeta.

Para ello lo que haremos será crear una carpeta en nuestra maquina master con un par de ficheros de prueba. La ruta de dicha carpeta en la maquina será **/home/Ubuntu/Practica2/FileManagerDir**.

```
:~/Practica2/FileManagerDir $ ls  
test1.txt test2.txt
```

Para conseguir tener una carpeta compartida entre todos los pods tendremos que usar un recurso llamado volúmenes. Un volumen es un directorio accesible para uno o varios contenedores.

Montaje de Volúmenes

Montaremos los volúmenes en los yaml. Para definir volúmenes y usarlos

- **volumes:** Permite definir volúmenes
- **hostPath:** Hace que el pod use un directorio ya existente en el nodo
- **path:** Ruta del directorio existente
- **type: Directory :** Se asegura que la ruta sea un directorio
- **volumeMounts:** Lista de volúmenes que usa el contenedor
- **name:** nombre del volumen
- **mountPath:** Directorio que ve el volumen en el contenedor

Ahora tenemos que añadir este volumen a los servidores para que estos puedan acceder a él. Para ello tendremos que modificar el deployment del servidor.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: server
spec:
  replicas: 3
  selector:
    matchLabels:
      app: server
  template:
    metadata:
      labels:
        app: server
    spec:
      containers:
      - name: server
        image: docker.io/19peirooo/server:latest
        ports:
        - containerPort: 32001
        volumeMounts:
        - name: shared-fm-dir
          mountPath: /app/FileManagerDir
      volumes:
      - name: shared-fm-dir
        hostPath:
          path: /home/ubuntu/Practica2/FileManagerDir
          type: Directory
```

Probaremos que sigue funcionando

```
root@client-65b494589b-jndxk:/app# ./clientFileManager 44.215.164.220 32002
Enter command:
ls
Listing files in local path
"./test2.txt"
"./test_local.txt"
"./clientFileManager"

Enter command:
upload test_local.txt
Coping file test_local.txt in to the FileManager path
Reading file: test_local.txt 0 bytes

Enter command:
lls
Listing files fileManager path
FileManagerDir/test2.txt
FileManagerDir/test1.txt
FileManagerDir/test_local.txt

Enter command:
```

Configuración Avanzada 2

Para la siguiente configuración, cogeremos lo ha implementado, pero ahora podemos usar varios nodos para los servidores. Al ocurrir esto, no podemos utilizar hostpath como antes e implementaremos un disco duro NFS.

NFS

NFS es un sistema de archivos en red que permite a múltiples nodos o pods compartir el mismo directorio remoto, haciendo posible que todos vean y modifiquen los mismos archivos como si estuvieran en su propio sistema local.

Para poder usar NFS tenemos que abrir los puertos 111,2049 y 20048 en nuestro grupo de seguridad

Reglas de entrada (14)										
Q Buscar										
☐	Name	ID de la regla del gr...	Versión de IP	Tipo	Protocolo	Intervalo de puertos	Origen	Descripción		
☐	-	sgr-0c91836b7cc63744f	IPv4	TCP personalizado	TCP	6443	0.0.0.0/0	-		
☐	-	sgr-05f69a4d07b75958c	IPv4	TCP personalizado	TCP	10259	0.0.0.0/0	-		
☐	-	sgr-00429f758cfcc6bcb	IPv4	SSH	TCP	22	0.0.0.0/0	-		
☐	-	sgr-07d0fb7c3013cb3a3	IPv4	TCP personalizado	TCP	30000 - 32767	0.0.0.0/0	-		
☐	-	sgr-046189c03111194db	IPv4	TCP personalizado	TCP	2379 - 2380	0.0.0.0/0	-		
☐	-	sgr-0f6370ab8d12f904	IPv4	NFS	TCP	2049	0.0.0.0/0	-		
☐	-	sgr-089883946e457846c	IPv4	TCP personalizado	TCP	8080	0.0.0.0/0	-		
☐	-	sgr-004a42823ac0ef294	IPv4	UDP personalizado	UDP	20048	0.0.0.0/0	-		
☐	-	sgr-083b8d93d81505a90	IPv4	UDP personalizado	UDP	111	0.0.0.0/0	-		
☐	-	sgr-059a254616a81a5eb	IPv4	TCP personalizado	TCP	10250	0.0.0.0/0	-		
☐	-	sgr-0f2dec3c6bbddcc478	IPv4	UDP personalizado	UDP	2049	0.0.0.0/0	-		
☐	-	sgr-0ba4f1d99b92dd368	IPv4	TCP personalizado	TCP	111	0.0.0.0/0	-		
☐	-	sgr-05d61a778524e047a	IPv4	TCP personalizado	TCP	10257	0.0.0.0/0	-		
☐	-	sgr-0bb3ac8da42092412	IPv4	TCP personalizado	TCP	20048	0.0.0.0/0	-		

Montaje Servidor NFS

Tenemos que instalar NFS en el nodo master ejecutando el siguiente comando: **sudo apt-get install -y nfs-common nfs-kernel-server** haciendo que esta maquina sea nuestro servidor NFS.

Después, crearemos la carpeta compartida usando el comando **sudo mkdir -p /var/nfs/FileManagerDir**. Esta carpeta será la que exportaremos a los clientes modificando el fichero **/etc/exports** y aplicaremos la configuración usando **sudo exportfs -a**

```
GNU nano 6.2 /etc/exports *
# /etc/exports: the access control list for filesystems which may be exported
# to NFS clients. See exports(5).
#
# Example for NFSv2 and NFSv3:
# /srv/homes hostname1(rw,sync,no_subtree_check) hostname2(ro,sync,no_subtree_check)
#
# Example for NFSv4:
# /srv/nfs4 gss/krb5i(rw,sync,fsid=0,crossmnt,no_subtree_check)
# /srv/nfs4/homes gss/krb5i(rw,sync,no_subtree_check)
#
/var/nfs/FileManagerDir *(rw,sync,no_subtree_check,no_root_squash)
```

- *****: Permite acceder a todos los nodos
- **rw**: Permite lectura y escritura en los clientes
- **sync**: El servidor escribe los cambios inmediatamente
- **no_subtree_check**: Evita comprobaciones extra cuando se exporta un subdirectorio
- **no_root_squash**: Permite que el usuario root del cliente siga siendo root en el servidor

Finalmente reiniciaremos el servidor usando **sudo systemctl restart nfs-kernel-server**.

Una vez montado el servidor NFS, tendremos que instalar el cliente de NFS en los nodos slave usando **sudo apt-get install -y nfs-common**.

PersistentVolume

Para poder usar nfs, tendremos que configurar un PersistentVolume. Un PV es un recurso de kubernetes donde se describe donde esta el almacenamiento y como acceder a él. Para ello tendremos que definir 2 archivos yaml para definir el PV y un PVC.

Para montar este PV se tendrá en cuenta lo siguiente:

- Tendrá de almacenamiento 1Gi
- Se puede montar desde varios pods a la vez

El primer yaml es el que definirá el PV:

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: nfs-pv
spec:
  capacity:
    storage: 1Gi
  accessModes:
    - ReadWriteMany
  persistentVolumeReclaimPolicy: Retain
  nfs:
    path: /var/nfs/FileManagerDir
    server: 172.31.92.191
```

- **capacity.storage:** Se definirá el tamaño del almacenamiento
- **accessMode: ReadWriteMany:** Permite que varios nodos escriban a la vez
- **persistentVolumeReclaimPolicy: Retain:** Si borras el deployment, el contenido no se borra
- **nfs.path:** Directorio exportado de NFS
- **nfs.server:** IP privada del servidor NFS

Una vez creado el yaml, haremos un apply para meter el PV en kubernetes.

```
~/Practica2/kub $ kubectl apply -f nfs-pv.yaml
persistentvolume/nfs-pv created
~/Practica2/kub $ kubectl get pv
NAME      CAPACITY  ACCESS MODES  RECLAIM POLICY  STATUS   CLAIM  STORAGECLASS  REASON  AGE
nfs-pv    1Gi      RWX           Retain          Available                                46s
```

PersistentVolumeClaim

Un PersistentVolumeClaim es una solicitud de almacenamiento persistente hecha por los pods. Kubernetes lo asocia a un PersistentVolume que cumple los requisitos, permitiendo que los contenedores tengan acceso a datos que persisten, aunque el pod se elimine o reinicie.

Como mencionado anteriormente, tendremos que crear un archivo yaml definiendo el PVC:

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: nfs-pvc
spec:
  accessModes:
    - ReadWriteMany
  resources:
    requests:
      storage: 1Gi
```

Una vez creado el yaml, haremos el apply para meter el PVC en kubernetes:

```
~/Practica2/kub $ kubectl apply -f nfs-pvc.yaml
persistentvolumeclaim/nfs-pvc created
~/Practica2/kub $ kubectl get pvc
NAME      STATUS    VOLUME   CAPACITY   ACCESS MODES   STORAGECLASS   AGE
nfs-pvc   Bound     nfs-pv   1Gi        RWX                             4s
~/Practica2/kub $
```

Podemos ver que se ha creado correctamente ya que su status es bound significando que se ha enganchado al PV correctamente.

Modificación Servidor

Una vez creados el PV y el PVC, tendremos que modificar el servidor para que el volumen que use nuestro PV. Para ello, a la hora de montar el volumen, añadiremos el PVC.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: server
spec:
  replicas: 3
  selector:
    matchLabels:
      app: server
  template:
    metadata:
      labels:
        app: server
    spec:
      containers:
      - name: server
        image: docker.io/l9peirooo/server:latest
        ports:
        - containerPort: 32001
        volumeMounts:
        - name: nfs-fm-dir
          mountPath: /app/FileManagerDir
      volumes:
      - name: nfs-fm-dir
        persistentVolumeClaim:
          claimName: nfs-pvc
```

```
:~/Practica2/kub $ kubectl apply -f deployment-broker.yaml
deployment.apps/broker created
:~/Practica2/kub $ kubectl apply -f deployment-server.yaml
deployment.apps/server created
:~/Practica2/kub $ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
broker-994469d6d-zb4m9             1/1     Running   0           15s
server-5bc59cd444-94w97            1/1     Running   0           7s
server-5bc59cd444-mz6gj            1/1     Running   0           7s
server-5bc59cd444-nrmv7            1/1     Running   0           7s
:~/Practica2/kub $ kubectl apply -f deployment-client.yaml
deployment.apps/client created
:~/Practica2/kub $ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
broker-994469d6d-zb4m9             1/1     Running   0           34s
client-65b494589b-mvwq2            1/1     Running   0           7s
server-5bc59cd444-94w97            1/1     Running   0           26s
server-5bc59cd444-mz6gj            1/1     Running   0           26s
server-5bc59cd444-nrmv7            1/1     Running   0           26s
:~/Practica2/kub $
```

Vamos a comprobar ahora si ha funcionado todo, para ello me meter en un servidor y creare un archivo, luego comprobare si ese archivo esta en otro servidor

```
~/Practica2/kub $ kubectl get pods
NAME                                READY    STATUS    RESTARTS   AGE
broker-994469d6d-zb4m9             1/1      Running   0           3m59s
client-65b494589b-mvwq2            1/1      Running   0           3m32s
server-5bc59cd444-94w97            1/1      Running   0           3m51s
server-5bc59cd444-mz6gj            1/1      Running   0           3m51s
server-5bc59cd444-nrmv7            1/1      Running   0           3m51s
~/Practica2/kub $ kubectl exec -ti server-5bc59cd444-94w97 -- bash
root@server-5bc59cd444-94w97:/app# cd FileManagerDir/
root@server-5bc59cd444-94w97:/app/FileManagerDir# ls
root@server-5bc59cd444-94w97:/app/FileManagerDir# touch archivo_prueba_pod1.txt
root@server-5bc59cd444-94w97:/app/FileManagerDir# echo "Este texto ha sido creado en server-5bc59cd444-94w97" > archivo_prueba_pod1.txt
root@server-5bc59cd444-94w97:/app/FileManagerDir# ls -l
total 4
-rw-r--r-- 1 root root 53 Nov 27 19:04 archivo_prueba_pod1.txt
root@server-5bc59cd444-94w97:/app/FileManagerDir# exit
exit
~/Practica2/kub $ kubectl exec -ti server-5bc59cd444-mz6gj -- ls /app/FileManagerDir
archivo_prueba_pod1.txt
~/Practica2/kub $
```

Comprobaremos también el flujo normal del programa

```
~/Practica2/kub $ kubectl exec -ti client-65b494589b-mvwq2 -- bash
root@client-65b494589b-mvwq2:/app# ls
clientFileManager
root@client-65b494589b-mvwq2:/app# ./clientFileManager 44.215.164.220 32002
Enter command:
ls
Listing files in local path
"./clientFileManager"

Enter command:
lls
Listing files fileManager path
FileManagerDir/archivo_prueba_pod1.txt

Enter command:
upload clientFileManager
Coping file clientFileManager in to the FileManager path
Reading file: clientFileManager 1223328 bytes
```